

Synapse Architecture: The Uncopyable Infrastructure Report

1. Executive Summary and Architectural Thesis

The Synapse project operates at the frontier of distributed systems engineering, aiming to solve a fundamental problem in software distribution: the unauthorized replication of execution contexts. In a standard microservices environment, a binary artifact—a Docker container or a compiled executable—is fungible. It can be copied, moved, and executed in unauthorized environments with minimal friction. The Synapse "Uncopyable" initiative seeks to negate this fungibility not through traditional Digital Rights Management (DRM), which is brittle and user-hostile, but through **Environmental Entanglement**.

Environmental Entanglement posits that a software system should not merely run *on* a host but should be inextricably *bound* to the specific, verifiable state of that host's kernel, memory subsystems, and governance policies. The current Synapse scaffolding provides a basic distributed systems framework but lacks the deep roots required for this entanglement. This report identifies and details the implementation of three critical missing components necessary to achieve this status: **eBPF Identity** (kernel-anchored verification), **Vector Memory** (context-dependent state retrieval), and **WASM Governance** (sandboxed, policy-driven logic).

The technical analysis that follows provides an exhaustive roadmap for building these components using the Rust ecosystem. It leverages `aya-rs` for kernel-space programming, `lancedb` and `candle` for semantic memory, and `wasmtime` with the Component Model for governance. The implementation plan addresses specific challenges identified in the research, such as cross-platform development (Windows/macOS vs. Linux eBPF), zero-copy deserialization alignment, and the integration of asynchronous runtimes across disparate technology stacks.

2. eBPF Identity: Kernel-Anchored Security and Observability

The first pillar of the uncopyable architecture is **eBPF Identity**. In traditional container orchestration (Kubernetes, Docker), identity is often injected as an environment variable or a mounted secret. These are user-space artifacts that can be trivially copied. To create an uncopyable system, identity must be derived from the immutable properties of the execution environment itself, observed and enforced by the operating system kernel.

2.1 Architectural Theory: Verification by Observation

The core philosophy of eBPF Identity is "verification by observation." Rather than an application presenting a credential to the network, the kernel observes the application's behavior, its lineage, and its resource utilization to assign a trusted identity. This prevents identity spoofing because the kernel structures that define a process (such as `task_struct`) cannot be forged from user space without root privileges, and even then, eBPF programs running in the kernel provide a higher privilege vantage point to detect anomalies.

This architecture relies on two primary mechanisms:

1. **LSM (Linux Security Module) Hooks:** These hooks act as a reference monitor, intercepting critical system operations like process allocation (`task_alloc`) or socket connections (`socket_connect`). By attaching an eBPF program here, the system can enforce that only processes with a specific cryptographic signature or running within a verified cgroup hierarchy can exist or communicate.¹
2. **Traffic Control (TC) and Socket Filters:** These attach to the network interface or cgroup sockets to tag or drop packets based on the kernel-verified identity of the sender, creating a "network diode" that physically prevents unauthorized binaries from communicating.³

2.2 The Aya-rs Framework and Rust Integration

The implementation strategy utilizes aya-rs, a Rust-native eBPF library. Unlike previous frameworks like BCC (BPF Compiler Collection) which required Clang/LLVM on the target machine for runtime compilation, or libbpf which relies on C tooling, Aya allows for a pure Rust workflow.⁵ This is critical for Synapse, as it allows the sharing of types and logic between the kernel-space verifier and the user-space application, reducing serialization errors and logical mismatches.

2.2.1 Workspace Scaffolding and Kernel Bindings

To support this integration, the project structure must be rigorously defined to separate kernel-space code (which must be `no_std`) from the user-space loader.

Workspace Structure:

```
synapse/
├── Cargo.toml # Workspace root
├── synapse-core/ # User-space application (Host)
│   ├── src/
│   └── Cargo.toml
├── synapse-ebpf/ # Kernel-space eBPF programs
│   ├── src/
│   └── Cargo.toml # Dependencies: aya-ebpf, network-types
└── xtask/ # Build automation and asset generation
```

A critical implementation detail often overlooked is the generation of kernel bindings. eBPF programs access raw kernel structures. The aya-tool must be employed to generate these bindings (e.g., `task_struct`, `sock`) from the host's vmlinux or BTF (BPF Type Format) data.⁷

Binding Generation Logic:

The xtask binary should include a command to invoke aya-tool. This ensures that the developer can regenerate bindings when the target kernel version changes.

Rust

```
// xtask/src/codegen.rs
pub fn generate() -> Result<(), anyhow::Error> {
    let names = vec!["task_struct", "vm_area_struct", "cgroup"];
    // Invoke aya-tool to generate bindings.rs in synapse-ebpf
    aya_tool::generate(names, Path::new("synapse-ebpf/src/bindings.rs"))
}
```

This generated file allows the Rust code to safely verify pointer dereferences in the kernel, a necessary step for inspecting cgroup paths to validate identity.⁸

2.3 Implementation of the Uncopyable Identity

The "uncopyable" property is enforced by checking the **Control Group (cgroup)** hierarchy. In a valid Synapse deployment, the binary runs inside a container orchestrated by a specific supervisor that places it in a unique cgroup (e.g., `/sys/fs/cgroup/synapse/verified/`).

2.3.1 The task_alloc LSM Hook

We attach an LSM program to `task_alloc`. This hook is triggered every time a process forks. The eBPF program inspects the parent process and the child process.

Kernel Logic:

1. **Intercept:** When `task_alloc` fires, access the `task_struct` of the process.
2. **Traverse:** Walk the pointers from `task_struct` -> `css_set` -> `cgroup` -> `kernfs_node` -> `name`. This traversal allows the eBPF program to read the cgroup name.
3. **Validate:** Compare the cgroup path against a compile-time hash of the authorized Synapse environment.
4. **Tag:** If the path matches, add the PID to a BPF Map (`TRUSTED_PIDS`). If it does not match, the process is tracked as "untrusted."

This creates a state where a copied binary running on a standard developer machine (e.g., in a default Docker container or bare metal) will fail the cgroup check. The eBPF program will not

tag it as trusted, effectively isolating it.¹

2.3.2 Network Enforcement via cgroup_skb

The identity established by the LSM hook is enforced at the network layer using a cgroup_skb program. This program attaches to the root cgroup and filters all egress traffic.

Mechanism:

The program retrieves the PID of the socket attempting to send data. It performs a lookup in the TRUSTED_PIDS map populated by the task_alloc hook.

- **Match Found:** The packet is allowed to proceed.
- **No Match:** The packet is dropped immediately (return 0).

This ensures that even if the binary is executed, it cannot exfiltrate data or participate in the cluster unless it is in the correct cgroup environment.⁴

2.4 Addressing the Cross-Platform Build Challenge

A significant hurdle identified in the research is the platform dependency of eBPF. eBPF is a Linux kernel technology. However, development teams often use macOS or Windows. A rigid requirement for Linux would hinder velocity. To resolve this, Synapse must implement a **Hybrid Build Strategy**.¹⁰

2.4.1 The Mock Identity Pattern

We introduce a trait-based abstraction for the identity provider in the synapse-core crate.

Rust

```
// synapse-core/src/identity/mod.rs
#[async_trait]
pub trait IdentityProvider {
    async fn verify_pid(&self, pid: u32) -> bool;
    async fn attach(&self) -> Result<(), Error>;
}
```

Conditional Compilation:

Using Cargo's target-specific dependencies, we load different implementations based on the OS.

Linux Implementation (src/identity/ebpf.rs):

This module is compiled only on Linux. It uses aya to load the actual BPF objects and interact with the kernel maps.

Rust

```
#[cfg(target_os = "linux")]
impl IdentityProvider for EbpIdentity {
    // Loads synapse-ebpf ELF, attaches hooks, manages BPF Maps
}
```

Mock Implementation (src/identity/mock.rs):

This module is compiled on macOS/Windows. It simulates the behavior of the eBPF layer using standard user-space structures (e.g., a DashMap tracking PIDs). This allows developers to write and test the upper layers of the application (Vector Memory, Governance) without needing a full Linux VM for every iteration.¹³

2.4.2 Docker-Based Compilation for Artifacts

While the user-space code can be mocked, the eBPF object files (.o) must eventually be compiled. For non-Linux developers, we implement a custom cargo runner in xtask that spins up a Docker container solely for the purpose of compiling the synapse-ebpf crate using the bpf-linker.

Build Workflow:

1. Developer runs cargo xtask build-ebpf.
2. xtask checks host OS.
3. If Linux: Runs cargo build directly.
4. If macOS/Windows:
 - Mounts the source directory into a Docker container (image: ghcr.io/aya-rs/aya-bpf-builder).
 - Executes the build inside the container.
 - Outputs the .o file to the host's target/ directory.

This ensures that the "Uncopyable" artifact—the eBPF bytecode—can be generated reproducibly regardless of the developer's machine.¹²

3. Vector Memory: Contextual State and High-Dimensional Retrieval

The second pillar of the architecture is **Vector Memory**. In an "uncopyable" system, access to data is not granted by static keys but by semantic context. The application "remembers" events based on their similarity to the current state. This requires a shift from standard

relational databases to a high-performance vector store embedded directly within the application binary.

3.1 Architectural Theory: Semantic Persistence

Traditional systems use explicit addressing (e.g., GET /users/123). Synapse's Vector Memory uses implicit addressing (e.g., GET context where embedding ~= current_state). This prevents data scraping because the "keys" are high-dimensional vectors (e.g., 384 or 768 dimensions) derived from the runtime memory of the eBPF-verified process. An attacker copying the database file cannot query it meaningfully without the proprietary embedding generation logic running inside the verified binary.

The architecture demands three performance characteristics:

1. **Embedded Latency:** No network hops to a database server; the DB must run in-process.
2. **Streaming Ingestion:** The ability to absorb high-velocity logs from the eBPF layer.
3. **Zero-Copy Access:** Retrieving complex context structures without the CPU overhead of deserialization.

3.2 LanceDB and Arrow Integration

We select **LanceDB** as the storage engine. Unlike external services (Pinecone, Qdrant), LanceDB can run embedded in Rust, storing data in the Lance columnar format (based on Apache Arrow). This allows for SIMD-accelerated scanning and zero-copy reads.¹⁵

3.2.1 Streaming Write Architecture

Synapse generates a continuous stream of operational events. Writing these one-by-one to disk is inefficient. We utilize LanceDB's support for Arrow RecordBatch streams to implement a buffered write system.

Implementation Logic:

We define a `MemoryManager` actor in Tokio. It holds a `Vec<LogEvent>` buffer.

1. **Ingest:** Events arrive via an `mpsc::channel`.
2. **Buffer:** Events are pushed to the vector.
3. **Flush Condition:** When the buffer reaches a threshold (e.g., 1024 events) or a timeout (1s) occurs.
4. **Conversion:** The buffer is converted into an Arrow RecordBatch. This is a columnar transposition—all timestamps in one array, all embeddings in another.
5. **Write:** The batch is sent to LanceDB via `Table::add(stream)`.

This approach aligns with the research on LanceDB's `write_fragments` and stream APIs, ensuring that I/O operations are batched and optimized for the underlying columnar format.¹⁷

3.3 Internal Embedding Generation with Candle

To ensure the system is uncopyable, the logic for generating vector embeddings must be internal. Relying on an API (like OpenAI) creates an external dependency and a reproducible interface. By embedding the model weights inside the container and running inference locally, we bind the data schema to the specific model version in the binary.

We utilize **Candle**, a minimalist ML framework for Rust.

Optimization Strategy:

ML inference is CPU intensive. Running it on the Tokio async runtime thread will block the event loop, causing network jitter.

- **Solution:** Use `tokio::task::spawn_blocking`.
- **Model:** Use a quantized model (e.g., all-MiniLM-L6-v2 quantized to q4_0) to reduce memory footprint and increase speed.
- **Loading:** Load weights from `.safetensors` files using memory mapping (`unsafe { MmapOptions::new().map(&file) }`) to minimize startup time.¹⁹

Rust

```
// synapse-memory/src/embedder.rs
pub async fn embed(&self, text: String) -> Result<Vec<f32>> {
    let model = self.model.clone();
    // Offload the heavy matrix multiplication to a blocking thread
    tokio::task::spawn_blocking(move |

| {
    let tokens = model.tokenizer.encode(text, true)?;
    let embeddings = model.forward(&tokens)?;
    Ok(normalize(embeddings))
}).await?
}
```

3.4 Zero-Copy Deserialization with Rkyv

When the Governance layer (WASM) requests context, the Memory system must return complex structures (e.g., a history of recent security violations). Using `serde_json` or `bincode` involves copying bytes from the database cache to a new struct, decoding fields one by one. This is too slow for the hot path of a security monitor.

We utilize **Rkyv**, a zero-copy deserialization framework. Rkyv guarantees that the in-memory representation of a struct is byte-identical to its serialized form.²⁰

3.4.1 The Alignment Challenge

A critical detail identified in the research is **Memory Alignment**. CPUs cannot efficiently access multi-byte types (like u64 or f32) at arbitrary memory addresses; they must be aligned to 4 or 8 bytes. rkyv enforces this strictly. Standard Vec<u8> buffers from a database read are not guaranteed to be aligned.

Implementation with AlignedVec:

We must wrap the data retrieval from LanceDB to use rkyv::AlignedVec.

Rust

```
use rkyv::{Archive, Serialize, Deserialize, AlignedVec};

#
#[archive(check_bytes)]
#[repr(C)] // Ensure C-compatible layout for stability
pub struct ContextEvent {
    timestamp: u64,
    source_pid: u32,
    details: String,
}

// Serialization
pub fn store_event(event: &ContextEvent) -> AlignedVec {
    rkyv::to_bytes::<_, 256>(event).expect("failed to serialize")
}

// Retrieval (Zero-Copy)
pub fn access_event(bytes: &[u8]) -> &ArchivedContextEvent {
    // This function casts the pointer. It is O(1).
    // Safety: Requires bytes to be aligned and validated.
    unsafe { rkyv::access_unchecked::<ContextEvent>(bytes) }
}
```

In the LanceDB schema, we store these serialized blobs in a binary column. Upon retrieval, we read the blob into an AlignedVec (if not already aligned by the Arrow buffer allocator) and cast it. This enables the WASM host to provide the guest with a pointer to memory that is instantly usable.²¹

4. WASM Governance: Sandboxed Policy Execution

The final pillar is **WASM Governance**. While eBPF provides the identity and Vector Memory provides the context, WASM Governance provides the *logic*. By implementing the policy engine as a WebAssembly component, we achieve a separation of concerns: the Synapse binary is merely a specialized runtime (Host), while the proprietary business logic runs as a secure Guest.

4.1 Architectural Theory: The Governance Sandbox

Standard applications compile logic directly into the binary. In Synapse, the logic is decoupled. The core binary provides "Mechanisms" (network access, db access), while the WASM module provides "Policy" (who can access what). This allows for:

1. **Dynamic Updates:** Policies can be patched without restarting the core binary.
2. **Capability Security:** The WASM module is sandboxed; it cannot access the file system or network unless the Host explicitly provides a function to do so.
3. **Cryptographic Binding:** The WASM module is encrypted and signed. The Host only decrypts and instantiates it if the eBPF Identity check confirms the environment is secure.

4.2 Wasmtime and the Component Model

We utilize **Wasmtime** as the runtime, specifically leveraging the **Component Model**. Unlike legacy WASM modules which expose low-level C-style functions, Components use high-level interfaces defined in **WIT (WebAssembly Interface Type)**.²³

4.2.1 Defining Interfaces with WIT

The WIT definition acts as the contract between the Synapse Host and the Governance Guest.

wit/governance.wit:

Code snippet

```
package synapse:governance;

interface context-access {
    // The Guest asks the Host to search vector memory
    // Returns a list of semantic matches
    search-memory: func(query: string) -> list<float32>;
}
```

```

interface network-control {
    // Guest instructs Host to allow/block a connection
    authorize-connection: func(ip: string, port: u16) -> bool;
}

world policy-engine {
    import context-access;
    import network-control;

    // The Host calls this when an event occurs
    export evaluate-event: func(event-type: string, payload: list<u8>) -> bool;
}

```

4.2.2 Host Implementation (The Middleware Pattern)

The Synapse Core acts as the middleware. It must implement the traits generated by bindgen! from the WIT file. This implementation bridges the gap between the sandboxed WASM and the actual Rust systems (LanceDB, Aya).

Rust

```

// synapse-core/src/governance/host.rs
use wasmtime::component::bindgen;
use crate::memory::MemoryStore;

// Generate the Rust traits from the WIT file
bindgen!({
    world: "policy-engine",
    path: "wit/governance.wit",
    async: true // Enable async support for non-blocking I/O
});

pub struct SynapseHost {
    // State shared with the WASM instance
    pub memory_store: Arc<MemoryStore>,
    pub db_pool: Arc<LanceDb>,
}

// Implement the import interface
#[async_trait]

```

```
impl synapse::governance::context_access::Host for SynapseHost {
    async fn search_memory(&mut self, query: String) -> wasmtime::Result<Vec<f32>> {
        // Bridge to the Vector Memory system
        let results = self.memory_store.semantic_search(&query).await?;
        Ok(results)
    }
}
```

This pattern allows the WASM guest to "think" (process logic) while delegating the "doing" (I/O) to the Host, which enforces the physical constraints.²⁵

4.2.3 Asynchronous Host Functions

A critical detail is the handling of async functions. Wasmtime supports async host functions, but the execution requires a carefully managed Store and Linker. The `bindgen!` macro with `async: true` generates traits that return `impl Future`. The Host's runtime (Tokio) drives these futures.

Resource Management:

The `Store<T>` in Wasmtime holds the state. Since we are using async networking and database access, the state `T` (our `SynapseHost` struct) must be thread-safe (Send + Sync). However, the Store itself is not thread-safe and must remain on a single thread or be moved linearly. To solve this, we instantiate a new Store for each governance request or use a pool of Stores for high throughput.²⁷

4.3 Secure Instantiation and Execution

The instantiation process ties the components together.

1. **Loader:** The Host loads the `governance.wasm` file.
2. **Decryption:** Ideally, this file is encrypted. The key is derived from the eBPF Identity Map (e.g., a hash of the allowed cgroup paths). If the eBPF system isn't active or the environment is wrong, the key derivation fails, and the WASM cannot be decrypted.
3. **Linkage:** The Linker connects the `SynapseHost` implementation to the WASM imports.
4. **Execution:** When an event occurs (e.g., `task_alloc` signal from eBPF), the Host calls `evaluate_event` on the WASM instance.

5. Implementation Roadmap and Integration Strategy

The following roadmap outlines the step-by-step construction of the Synapse system, prioritizing the resolution of dependencies and architectural foundations.

Phase 1: Foundation and Scaffolding (Weeks 1-3)

Objective: Establish the build system and cross-platform capabilities.

- **Action:** Create the Cargo workspace with core, ebpf, memory, and governance members.
- **Action:** Implement xtask for Docker-based eBPF compilation (solving the Windows/Mac build issue).
- **Action:** Define the IdentityProvider trait and the Mock implementation for local dev.
- **Deliverable:** A cargo build command that succeeds on all platforms, producing a binary that runs (mocked) on Windows and (natively) on Linux.

Phase 2: The Kernel Root (Weeks 4-6)

Objective: Implement eBPF Identity.

- **Action:** Generate vmlinux.h bindings using aya-tool.
- **Action:** Implement the task_alloc LSM hook to verify cgroup paths.
- **Action:** Implement the cgroup_skb hook to drop traffic from untrusted PIDs.
- **Deliverable:** A Linux binary that, when run outside the correct cgroup, cannot send network packets.

Phase 3: The Semantic Brain (Weeks 7-9)

Objective: Build Vector Memory.

- **Action:** Integrate lancedb and design the Arrow schema.
- **Action:** Implement the candle embedding engine with spawn_blocking offloading.
- **Action:** Build the streaming ingestion actor (RecordBatch buffering).
- **Action:** Implement rkyv serialization and the AlignedVec wrapper for zero-copy reads.
- **Deliverable:** A module that accepts log strings and persists them as searchable vectors.

Phase 4: The Governance Policy (Weeks 10-12)

Objective: Integrate WASM.

- **Action:** Define governance.wit.
- **Action:** Implement the SynapseHost adapter linking WASM imports to LanceDB and eBPF maps.
- **Action:** Compile a sample Rust policy to WASM using cargo component.
- **Deliverable:** A system where the eBPF layer triggers a WASM function, which queries the Vector Memory before authorizing an action.

Phase 5: The "Uncopyable" Lock (Weeks 13-14)

Objective: Bind the components.

- **Action:** Modify the WASM loader to require a decryption key derived from the eBPF TRUSTED_PIDS map state.
- **Action:** Harden the binary to panic if cgroup verification fails during startup.

- **Deliverable:** Final Release Candidate.

6. Conclusion

The Synapse project's transition to an "uncopyable" infrastructure relies on the seamless integration of **eBPF Identity**, **Vector Memory**, and **WASM Governance**. By moving identity verification to the kernel (LSM hooks), state to a semantic vector space (LanceDB/Candle), and logic to a governed sandbox (Wasmtime), the system creates a deep dependency on its deployment environment.

This architecture ensures that the binary is not a standalone product but a component of a verified whole. It effectively neutralizes the threat of binary theft, as the stolen artifact lacks the kernel-level roots, the semantic context, and the decrypted policy logic required to function. The roadmap provided herein leverages the strongest capabilities of the Rust ecosystem to deliver this secure, high-performance distributed system.

Works cited

1. Secure Namespaced Kernel Audit for Containers - Xueyuan Vanbastelaer, accessed on December 8, 2025, <https://www.vanbastelaer.com/publication/sabpf/sabpf.pdf>
2. LSM - Building eBPF Programs with Aya, accessed on December 8, 2025, <https://aya-rs.dev/book/programs/lsm>
3. cgroup_traffic - Rust - Docs.rs, accessed on December 8, 2025, https://docs.rs/cgroup_traffic
4. Cgroup SKB - Building eBPF Programs with Aya, accessed on December 8, 2025, <https://aya-rs.dev/book/programs/cgroup-skb>
5. Aya is an eBPF library for the Rust programming language, built with a focus on developer experience and operability. - GitHub, accessed on December 8, 2025, <https://github.com/aya-rs/aya>
6. Aya: your tRusty eBPF companion - Security Boulevard, accessed on December 8, 2025, <https://securityboulevard.com/2022/07/aya-your-trusty-ebpf-companion/>
7. Using aya-tool - Building eBPF Programs with Aya, accessed on December 8, 2025, <https://aya-rs.dev/book/aya/aya-tool>
8. Writing eBPF Kprobe Program with Rust Aya - Yuki Nakamura's Blog, accessed on December 8, 2025, <https://yuki-nakamura.com/2024/09/14/writing-ebpf-kprobe-program-with-rust-aya/>
9. SkBuffContext and IPv6 · Issue #82 · aya-rs/book - GitHub, accessed on December 8, 2025, <https://github.com/aya-rs/book/issues/82>
10. Cross-compile a Rust application from Linux to Windows - Stack Overflow, accessed on December 8, 2025, <https://stackoverflow.com/questions/31492799/cross-compile-a-rust-application-from-linux-to-windows>
11. Lab: Setting up a Rust

12. Docker environment for aya rust ebpf compilation - Stack Overflow, accessed on December 8, 2025,
<https://stackoverflow.com/questions/79313837/docker-environment-for-aya-rust-ebpf-compilation>
13. Conditionally compilation based on target OS - The Rust Programming Language Forum, accessed on December 8, 2025,
<https://users.rust-lang.org/t/conditionally-compilation-based-on-target-os/89119>
14. How to specify the target_os in `Cargo.toml`? - Stack Overflow, accessed on December 8, 2025,
<https://stackoverflow.com/questions/72990789/how-to-specify-the-target-os-in-cargo-toml>
15. Build a Fast and Lightweight Rust Vector Search App with Rig & LanceDB - DEV Community, accessed on December 8, 2025,
<https://dev.to/0thtachi/build-a-fast-and-lightweight-rust-vector-search-app-with-rig-lancedb-57h2>
16. Columnar File Readers in Depth: APIs and Fusion - LanceDB, accessed on December 8, 2025,
<https://lancedb.com/blog/columnar-file-readers-in-depth-apis-and-fusion/>
17. write_fragments in lance::dataset - Rust - Docs.rs, accessed on December 8, 2025, https://docs.rs/lance/latest/lance/dataset/fn.write_fragments.html
18. Custom Datasets for Efficient LLM Training Using Lance - LanceDB, accessed on December 8, 2025,
<https://lancedb.com/blog/custom-dataset-for-llm-training-using-lance/>
19. Building a High-Performance Text Embedding API with Rust, Axum, Candle and ONNX, accessed on December 8, 2025,
<https://dev.to/mayu2008/building-a-high-performance-text-embedding-api-with-rust-axum-and-onnx-12j4>
20. Zero-copy deserialization - rkyv, accessed on December 8, 2025,
<https://rkyv.org/zero-copy-deserialization.html>
21. Zero-copy (de)serialization | Hyper-Efficient Message Streaming at Laser Speed., accessed on December 8, 2025,
<https://iggy.apache.org/blogs/2025/05/08/zero-copy-deserialization/>
22. AlignedVec in rkyv::util - Rust - Docs.rs, accessed on December 8, 2025,
<https://docs.rs/rkyv/latest/rkyv/util/struct.AlignedVec.html>
23. wasmtime - Rust - Docs.rs, accessed on December 8, 2025,
<https://docs.rs/wasmtime>
24. Wasmtime - The WebAssembly Component Model, accessed on December 8, 2025,
<https://component-model.bytecodealliance.org/running-components/wasmtime.html>
25. Host in wasmtime::component::bindgen_examples::_2_world_exports, accessed on December 8, 2025,
https://docs.wasmtime.dev/api/wasmtime/component/bindgen_examples/_2_world_exports/my/project/host/trait.Host.html
26. wasmtime::component - Rust, accessed on December 8, 2025,

<https://docs.wasmtime.dev/api/wasmtime/component/index.html>

27. Func in wasmtime - Rust - Docs.rs, accessed on December 8, 2025,

<https://docs.rs/wasmtime/latest/wasmtime/struct.Func.html>

28. Config in wasmtime - Rust, accessed on December 8, 2025,

<https://docs.wasmtime.dev/api/wasmtime/struct.Config.html>